Document Editor System design

① Use Case

↳ Our editor currently supports text and image but it should be scalable so that it can support tables, videos, Fonts ... etc in future.

Notes :→

To approach any LLD problem we have 2 approach

① Top-Down Approach : →

→ Pehle sabse Top wala object banate hai aur fir uske andhar chote chote objects banate hai aur unki dependencies banate hai.

② Bottom-Up Approach :→

→ Pehle chote chote objects banate hai aur unki dependencies banate hai aur uske bad hum bade objects create karte hai aur fir uski dependency uske sath banate hai.

In most of cases, people use bottom-up approach.

Kuch questions me Top-down use karna better hota hai [we will see such Questions in future]

Here will be using Bottom-up

↳ So we have to create document-editor application.

① EK class bana do document editor.

② Socho ki document editor me kya kya hona chaiye

user texts aur images dal sakta hai editor me. So we will keep list of text & list of images but let's say customer ne pehle text dala fir image dala fir firse text dala .. so on

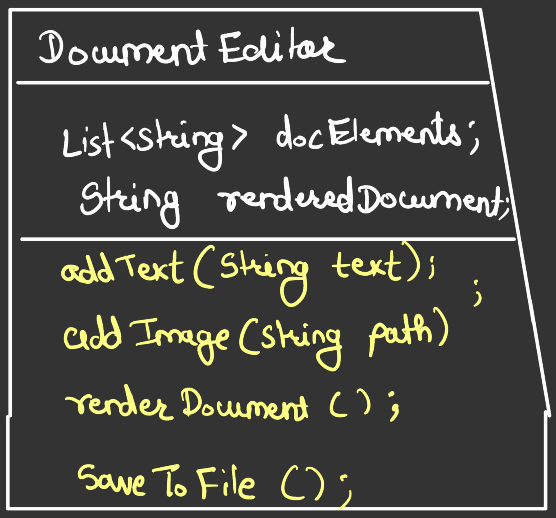
Also to hume order bhi maintain karna hoga.

So let's keep single list which will have text & image both.

So what data-type can we use to store both text & image ?

text → String

image → "Path" ⟶ String.



⊙ This is very bad design

→ Problems :→

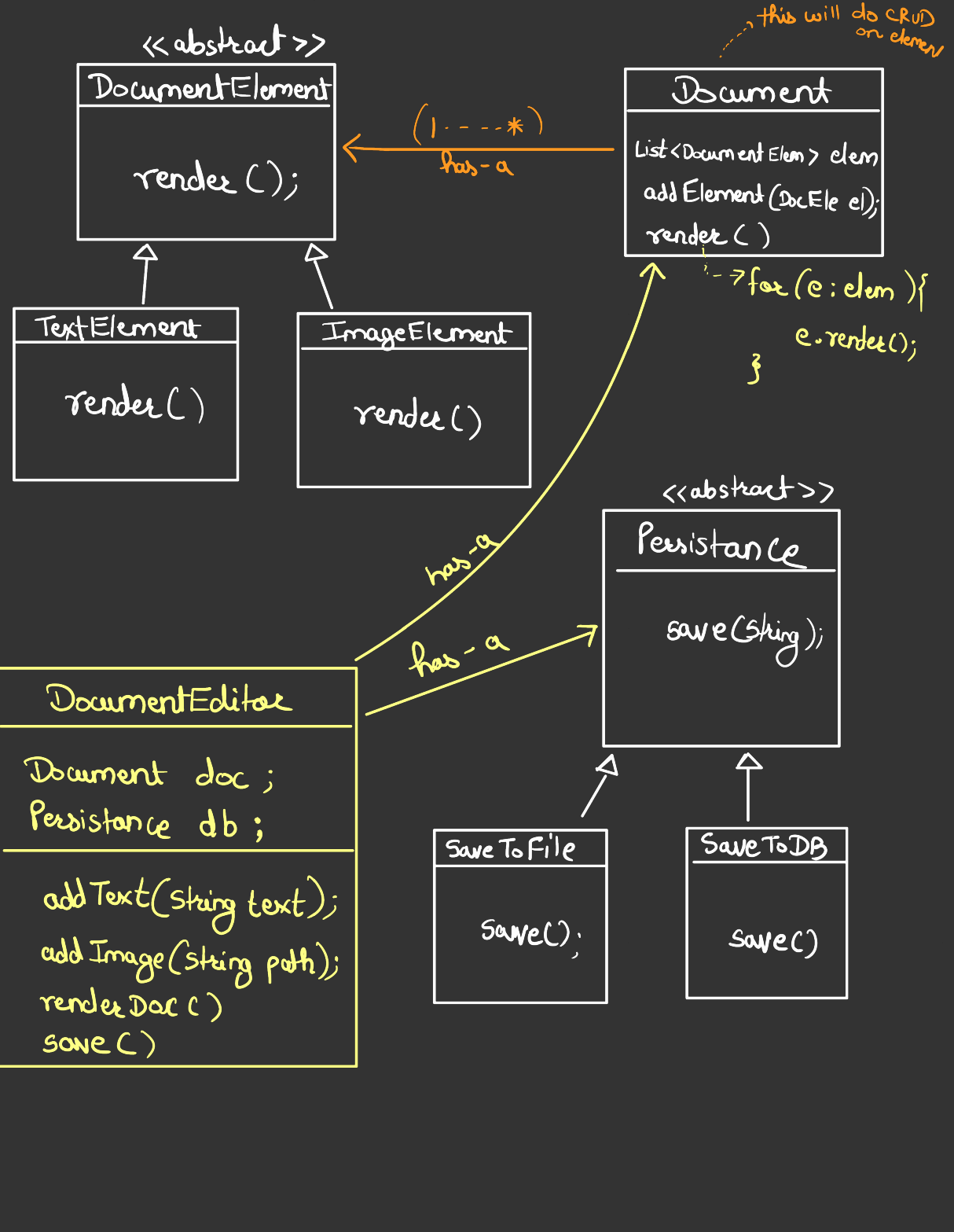
① This is breaking Single Responsibility Principal

② This is also breaking Open-Close Principal.

↳ Future me let's say hume table, videos... ka support dena hai to hume class ko open karna padega aur new methods add karna padega.

③ Yaha par ek hi class hai isliye LSP, DIP & ISP is out of scope.

Now we have to design follow SOLID principals.


this will do CRUD on element
<<abstract>>
DocumentElement
render();
(1 - - - *)
has-a
Document
List<DocumentElem> elem
addElement(DocEle el);
render()
for(e : elem){
e.render();
}
TextElement
render()
ImageElement
render()
<<abstract>>
Persistance
save(String);
has-a
has-a
DocumentEditor
Document doc;
Persistance db;
addText(String text);
addImage(String path);
renderDoc()
save()
SaveToFile
save();
SaveToDB
save()

Now difference in new & old design

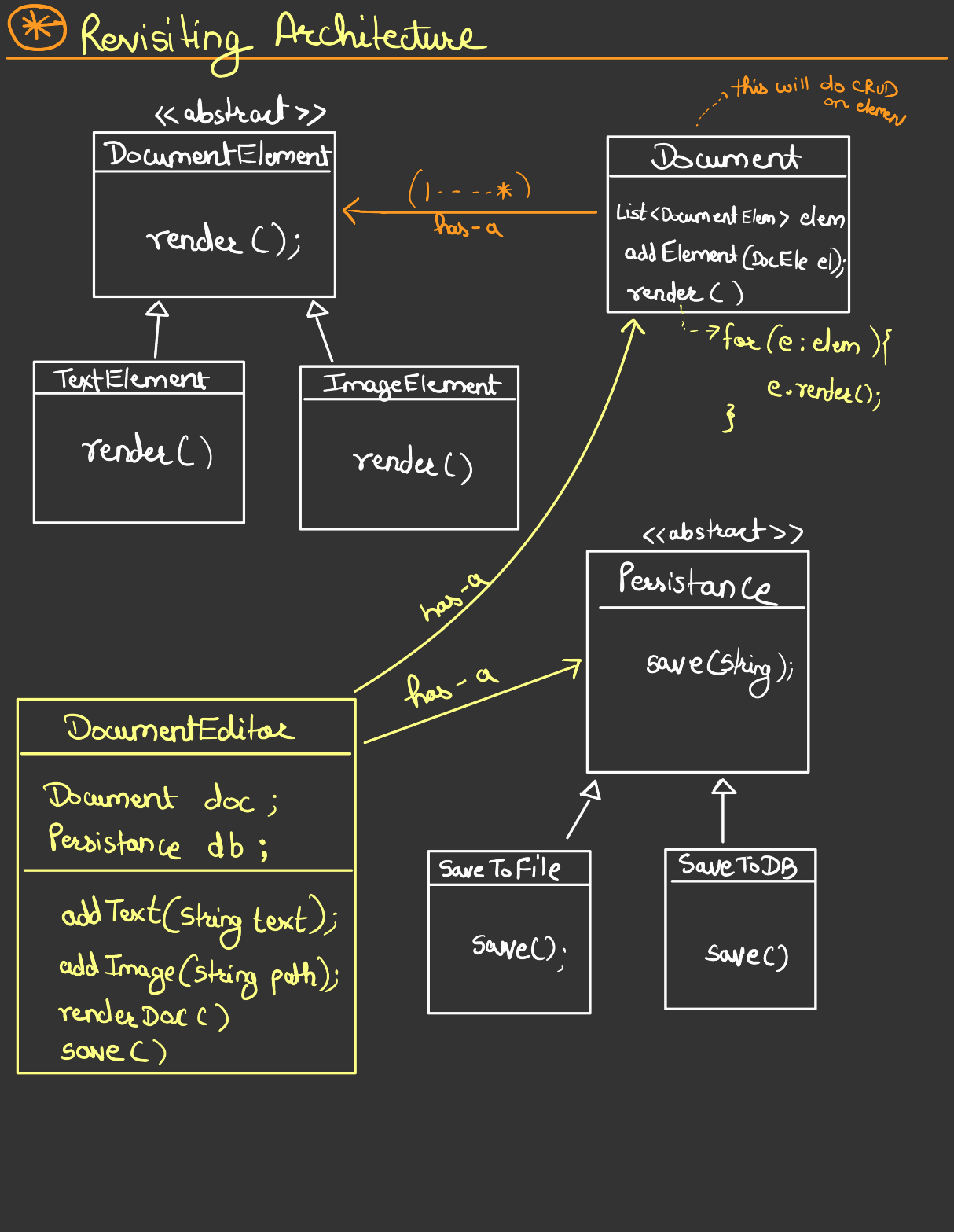
Old design me "DocumentEditor" class was responsible for executing all the functions in it.

New design me, functions sare same hai "DocumentEditor" class me jo old design me the but new design me DoumentEditor class sare method khud execute nahi karta, vo delegate karta hai respective class ko.

---

* Client will only Contact with DoumentEditor

Revisiting Architecture



↳ It is following SRP ✓

↳ It is following OCP ✓

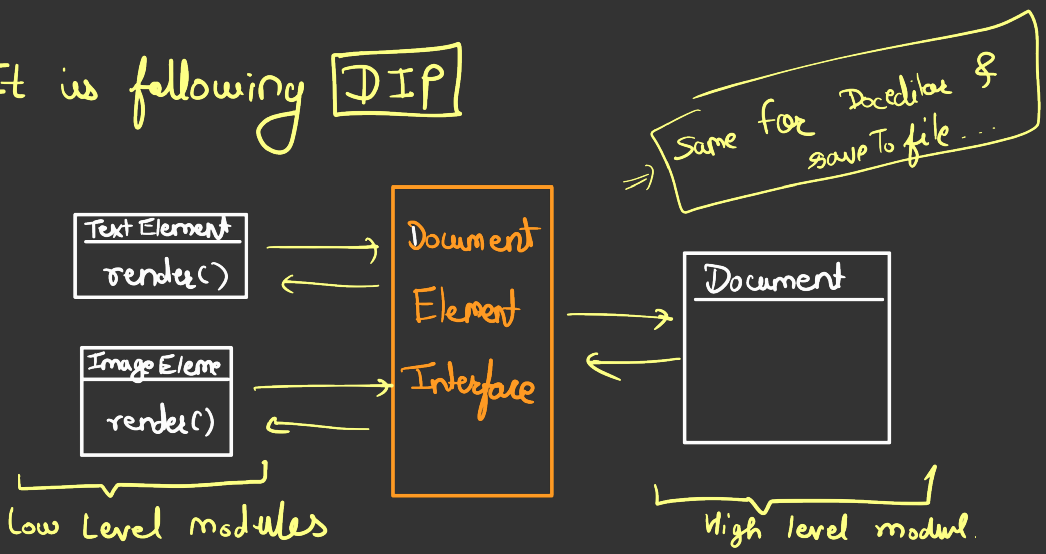
↳ It is following LSP ✓

DocumentElement class ke object ki jagah TextElement or ImageElement pass kar sakte hai.

↳ It is following ISP ✓

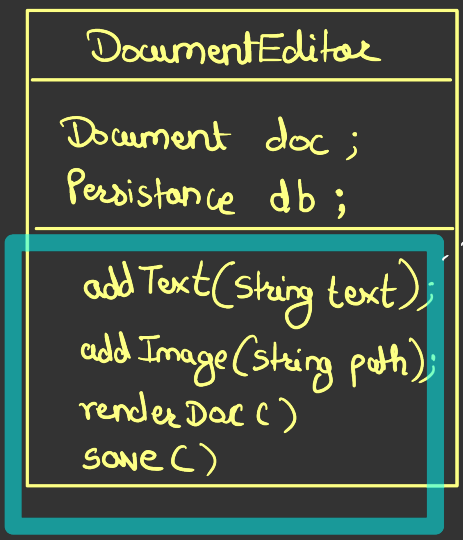
→ Interfaces me koi aisi method nahi hai jo child ko jabardasti implement karni pad rahi hai.

↳ It is following DIP



Enhancements in our Design

1



even though it delegates the individual functionality to respective class but still DocumentEditor class should be capable of remembering all the methods & how to call a particular method

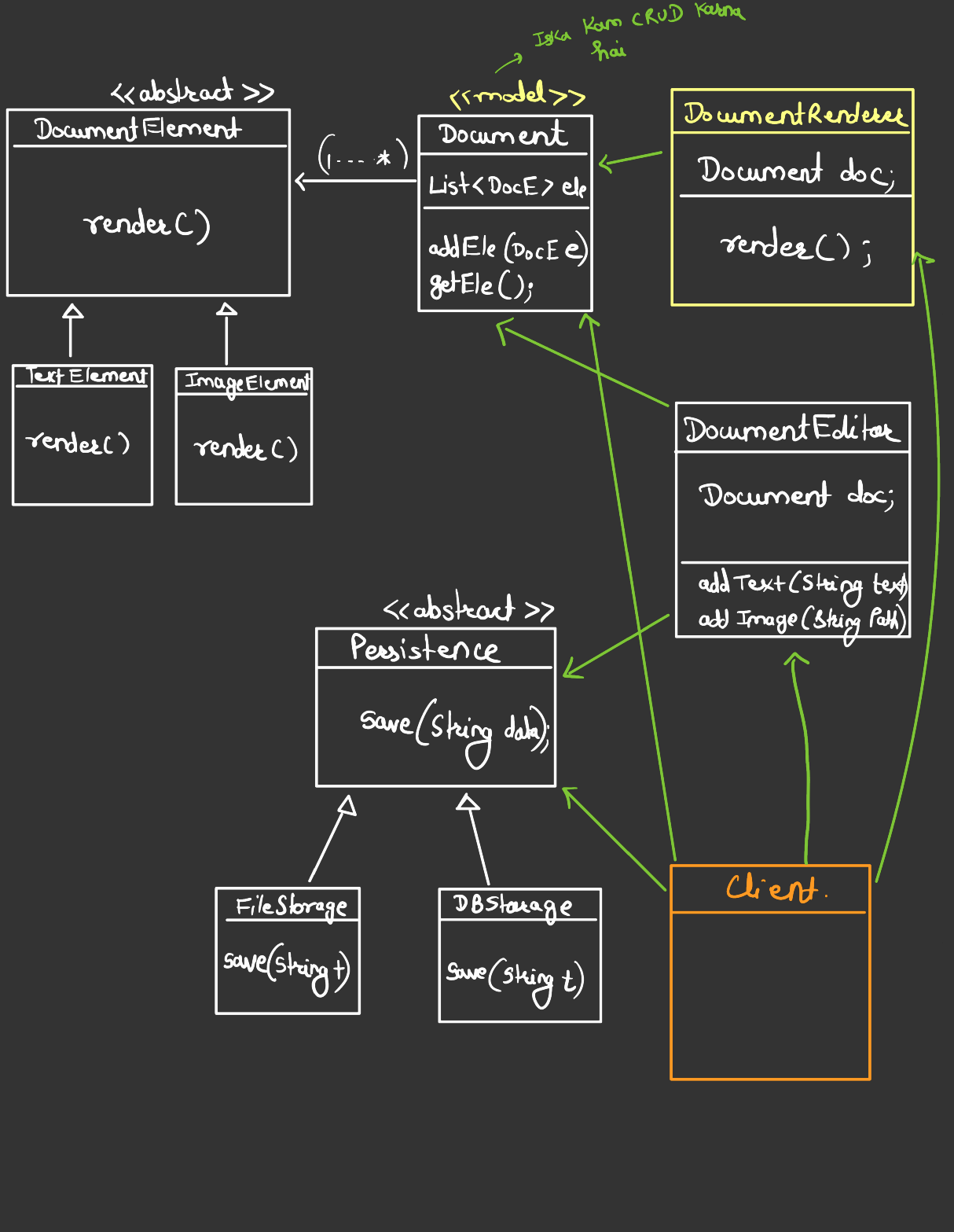
Eg:- Let's say, "Document" class se render() func^n hat gaya to "Document" editor se bhi hatana padega. Same for save() fun^c.

It is not directly violating "SRP".

↳ Ha bhi aur nahi bhi

It itself is not calling all the funcs.

It has to keep knowledge of each fun^c to call


Iska Kam CRUD Karna Hai
<<abstract>>
DocumentElement
render()
(1 ... *)
<<model>>
Document
List<DocE> ele
addEle(DocE e)
getEle();
DocumentRenderer
Document doc;
render();
TextElement
render()
ImageElement
render()
DocumentEditor
Document doc;
addText(String text)
addImage(String Path)
<<abstract>>
Pessistence
Save(String data);
FileStorage
save(String t)
DBStorage
Save(String t)
Client.

Now this design, lead to a new Problem called

"Principal of Least Knowledge".

It is a design Principal which says.

"You should always talk to your immediate Friends".

This is needed because otherwise Coupling increases.

* There is always a trade-off.